University of Hertfordshire UH

School of Physics, Engineering and Computer Science

MSc Data Science Project
7PAM2002

Department of Physics, Astronomy, and Mathematics

Data Science FINAL PROJECT REPORT

Project Title:

Time Series Forecasting of Agricultural Production

Student Name and SRN:

Mani Deepak Soma

24095193

Supervisor: Puspa Raj Tiwari

Date Submitted: 21-04-2026

Word Count: 4970

Git hub Repository: https://github.com/Manideepak077/Time-Series-Forecasting-of-Agricultural-Production

DECLARATION STATEMENT

This report is submitted in partial fulfilment of the requirements for the degree of Master of Science **in Data Science** at the University of Hertfordshire.

I have read the detailed guidance to students on academic integrity, misconduct, and plagiarism information at Assessment Offences and Academic Misconduct, and understand the University process of dealing with suspected cases of academic misconduct and the possible penalties, which could include failing the project or course.

I certify that the work submitted is my own and that any material derived or quoted from published or unpublished work of other persons has been duly acknowledged. (Ref. UPR AS/C/6.1, section 7 and UPR AS/C/5, section 3.6)

I did not use human participants in my MSc project.

I hereby give permission for the report to be made available on module websites, provided the source is acknowledged.

Student Name printed: Mani Deepak Soma

Student Name Signature: Mani Deepak Soma

Student SRN number: 24095193

UNIVERSITY OF HERTFORDSHIRE

SCHOOL OF PHYSICS, ENGINEERING, AND COMPUTER SCIENCE

ABSTRACT

Predicting the agricultural production that are correct are of utmost importance in order to guarantee that there is sufficient food for everyone, to maintain the stability of commodity markets, and to assist in the management of resources based on evidence provided. Wheat is a staple crop that is grown in every part of the world, making it one of the most frequently cultivated. When it comes to the functioning of supply chains, the actions taken by the government, and the formation of trade agreements, the ability to accurately forecast the amount of wheat that will be grown each year has a direct impact.

To determine the accuracy of wheat production forecasts for India, the objective of this project is to utilize annual yield data from the FAOSTAT database, administered by the Food and Agriculture Organization of the United Nations (FAO, 2024). This study compares three distinct modeling approaches: a baseline, a traditional statistical model, ARIMA, and an LSTM deep learning methodology.

We utilised the root mean squared error, mean absolute percentage error, and mean absolute error to evaluate the overall performance of each model. The baseline model may show the worst performance imaginable, while the ARIMA model can show broad linear development patterns. The dynamic and non-linear nature of the collection, however, presents a challenge for it. It was demonstrated that deep learning architectures are preferable when it comes to learning challenging temporal correlations in agricultural production data by the fact that the LSTM model achieved the best results across all three specifications.

When it comes to agricultural forecasting tasks, such as estimating crop yields and making the most efficient use of resources, these results not only demonstrate how traditional statistical models can assist us in better understanding things, but they also add to the growing body of evidence that machine learning can be effective in these kinds of activities.

Content

1: INTRODUCTION 5
- 1.1 Background of the Study
- 1.2 Problem Statement
- 1.3 Aim of the Study
- 1.4 Objective
- 1.5 Scope of the Study

2: LITERATURE COMPARISON 6
- 2.1 Research Gap

3: METHODOLOGY 7
- 3.1 Collecting Data
- 3.2 Ethical Considerations
- 3.3 Data Preprocessing
- 3.4 Exploratory Data Analysis 8
- 3.5 ADF/ACF/PACF Analysis 9
- 3.6 Training and Testing
- 3.7 Models 10
  - 3.6.1 Baseline forecast
  - 3.6.2 ARIMA forecast 12
  - 3.6.3 LSTM forecast 14
- 3.8 Evaluation Metrics 16

4: RESULTS 18

5: CONCLUSION 19
- 5.1 Conclusion
- 5.2 Limitations
- 5.3 Future work

REFERENCES 20

APPENDIX 21

1: INTRODUCTION

1.1 Background of the Study

Agriculture is a critical component of the economy, allowing people to exist and thrive. A huge portion of the world's population benefits from it in terms of food, raw resources, and employment. Agriculture contributes significantly to the Gross Domestic Product (GDP) of many emerging and developed countries and is critical to people's survival.
Wheat is among the most important grain crops in the world. Billions of people eat it as their primary diet. Wheat demand continues to rise as the population grows, cities expand, and people's eating patterns shift. As a result, being able to precisely forecast how much wheat will be cultivated is critical for keeping food secure, markets stable, and distribution systems running smoothly. Many factors can influence agricultural production, including weather, soil condition, government regulations, new technologies, and market shifts. This makes it difficult to forecast what will occur. Time-series analysis is a methodical approach to modeling historical data and forecasting future patterns based on previous events.

1.2 Problem Statement

Agriculture is one of most essential aspects of the economy that helps people live and flourish. A lot of people in the world gain food, raw materials, and jobs from it. In many developing and developed countries, agriculture is a large portion of the gross domestic product (GDP) and is very important for keeping people alive. Wheat is one of the most important grain crops in the world. It is the major food for billions of people. The demand for wheat is still rising up because more people are moving to cities, cities are getting bigger, and people are altering the way they eat. So, it is very important to be able to precisely anticipate how much wheat will be cultivated to keep food safe, maintain markets stable, and make sure that distribution systems work smoothly. A lot of elements can effect farming, like the weather, the soil quality, government rules, new technologies, and changes in the market. This makes it challenging to guess what will happen. Time-series analysis is a methodical approach to examining historical data to forecast future trends based on past occurrences.

1.3 Aim of the Study

The purpose of this research is to forecast wheat output by utilizing time-series models and to assess the efficiency of statistical and deep learning techniques.

1.4 Objective

- To gather and prepare data related to agricultural productivity.
- The goal is to conduct exploratory data analysis to uncover trends and patterns.
- The goal is to implement forecasting models, including Naïve, ARIMA, and LSTM.
- The objective is to assess the model's performance utilising suitable metrics.
- To see how well different models work.

1.5 Scope of the Study

This research centers on predicting wheat production utilising annual data from a certain country. The research is confined to univariate time-series modeling, which means it only looks at past production values and not any outside factors.

2: LITERATURE COMPARISON

Time-series forecasting has been thoroughly examined in both statistics and machine learning fields. Conventional models like ARIMA are often utilized for their simplicity and interpretability. These models depend on assumptions of linearity and stationarity, rendering them appropriate for datasets exhibiting steady patterns.

Nonetheless, agricultural data frequently display nonlinear patterns and complex relationships that classic models cannot sufficiently represent. This constraint has prompted the advancement of sophisticated machine learning methodologies, especially deep learning models.

LSTM networks, a subtype of recurrent neural networks (RNNs), were created to rectify the limitations of traditional RNNs in handling long-term dependencies. LSTM models employ memory cells and gating mechanisms to retain crucial information across time, making them highly successful for sequential data.

A multitude of studies has demonstrated the superiority of LSTM models over traditional methods in time-series forecasting challenges. LSTM models have been successfully employed in agriculture to predict crop yields, weather patterns, and production trends.

Model	**Study**	**Key Findings**	**Support for Project**
Baseline (Naïve)	Makridakis et al. (1998)	Simple forecasting works well for short-term stable data.	Provides a benchmark to Compare advanced models.
ARIMA	Box & Jenkins (1976)	The ARIMA model looks at time series data to find patterns and trends.	Justifies the use of ARIMA for agricultural production forecasting.
LSTM	Yoo & Oh (2020)	LSTM captures nonlinear patterns and long-term dependencies.	Supports the use of LSTM for complex data.

2.1 Research Gap

There are still gaps in the literature, even though predicting techniques have come a long way:

- Comparative analyses between ARIMA and LSTM in agricultural forecasting are scarce.
- Insufficient focus on preprocessing methods.
- Not much usage of real-world datasets for testing

3: METHODOLOGY

3.1 Collecting Data

The Food and Agriculture Organization (FAO) of the United Nations operates the FAOSTAT database, which is a widely recognized source of data for this investigation. The agricultural data that FAOSTAT maintains is comprehensive, standardized, and accurate, spanning a wide range of nations, regions, and periods. Thus, it serves as an advantageous resource for the examination of time-series forecasting.

The collection comprises annual statistics on agricultural production for certain crops in different nations over a period of several years. The dataset usually informs the type of crop, the country or location it came from, the year it was made, and the amount of production measured in tonnes. These traits make it possible to examine past trends and patterns in agricultural output, which is necessary for developing models that forecast the future.

The data covers several years, which gives it enough time depth for time-series analysis. The fact that this information is longitudinal makes it easier to find patterns, seasonality, and changes in crop production over time. The dataset is also free to the public and follows open data principles, which makes the research more transparent and easier to repeat.

I obtained the dataset in CSV format and used Python packages like Pandas to import it into the working environment before analyzing it and looking at the data for the first time to get a sense of its structure, the types of variables it has, and whether it was comprehensive. Because real-world datasets often have errors, I did some preliminary checks to find missing values, duplicates, and possible outliers.

FAO collects data from national statistical bureaus and checks it for accuracy and consistency. But there may still be small disparities because different countries have different rules about how to report data. These problems were fixed at the data preprocessing stage to make sure the analysis was accurate.

The FAOSTAT dataset is a strong base for this study since it has high-quality, long-term agricultural data that can be used to create and test forecasting models like ARIMA and LSTM.

3.2 Ethical Considerations

The only data that is used in this project is economic information that is readily available to the public. Personal or identifying information is not included in this document. As a consequence of this, the terms of the GDPR that govern the handling of personal information do not apply to the data. This research does not require permission from the University of Hertfordshire Ethics Committee because it does not involve any surveys, interviews, or participants. Research and analysis by the general public is intended to be conducted using the data, which originates from reliable government sources. The project is conducted in accordance with the ethical principles and best research methods established by the university.

3.3 Preprocessing Data

Data preprocessing represents an essential phase in guaranteeing the integrity and dependability of the analysis. The subsequent steps were executed:

- Filtering the dataset to include only relevant variables (country, crop, production).
- Converting the dataset from wide format to long format.

- Extracting numeric year values from column names.
- Handling missing values by removing incomplete records.
- Removing invalid values such as zero or negative production.
- Converting the dataset into a time-series format.
- The training set and a testing set are created from data.
- Scaling data for LSTM model.
- Creating time-step sequences for deep learning.

3.4 Exploratory Data Analysis

Exploratory Data Analysis (EDA) was conducted to improve understanding the dataset and to identify underlying patterns, trends, and anomalies prior to model development. The analysis included statistical summaries and visual representations, such as line graphs, trend evaluations, and distribution assessments.

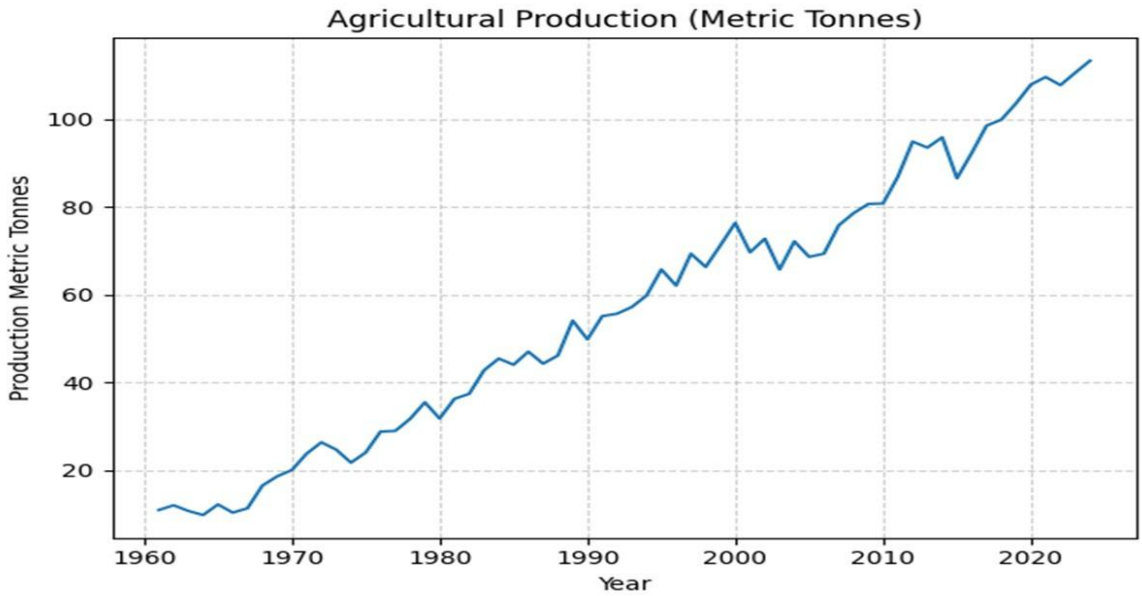

Figure 1: EXPLORATORY DATA ANALYSIS

The preliminary visualization of wheat production over time demonstrated a distinct increasing trajectory, signifying a consistent augmentation in agricultural yield throughout the examined duration. This expansion can be ascribed to innovations in agricultural technology, enhanced farming methodologies, and heightened demand. Notwithstanding this general upward trend, significant variations were evident in specific years. These variances may be affected by external variables like unfavorable climatic conditions, alterations in governmental policy, economic influences, and disruptions in the supply chain.

A descriptive statistical analysis was performed to summarise key features of the dataset, including metrics such as mean, median, standard deviation, minimum, and maximum values. These measurements provided insights into the central tendency and variability of wheat production throughout time.

Subsequent investigation entailed examining for absent values and anomalies. All absent data points were addressed adequately through acceptable imputation methods or by eliminating inconsequential records to preserve data integrity. Outliers, if identified, were meticulously

analyzed to ascertain if they indicated authentic exceptional occurrences or data entry inaccuracies.
Stationarity tests were performed to evaluate the data's appropriateness for time-series modelling. The dataset was determined to be non-stationary, indicated by observable patterns and fluctuating variance across time.

3.5 ADF/ACF/PACF Analysis

This was additionally validated using statistical assessments, including the Augmented Dickey-Fuller Test. Consequently, differencing techniques were employed to convert the data into a stationary format, a fundamental prerequisite for models such as ARIMA.
The EDA method yielded significant insights into the dataset's structure and behaviour, guiding preprocessing and model selection for precise forecasting.

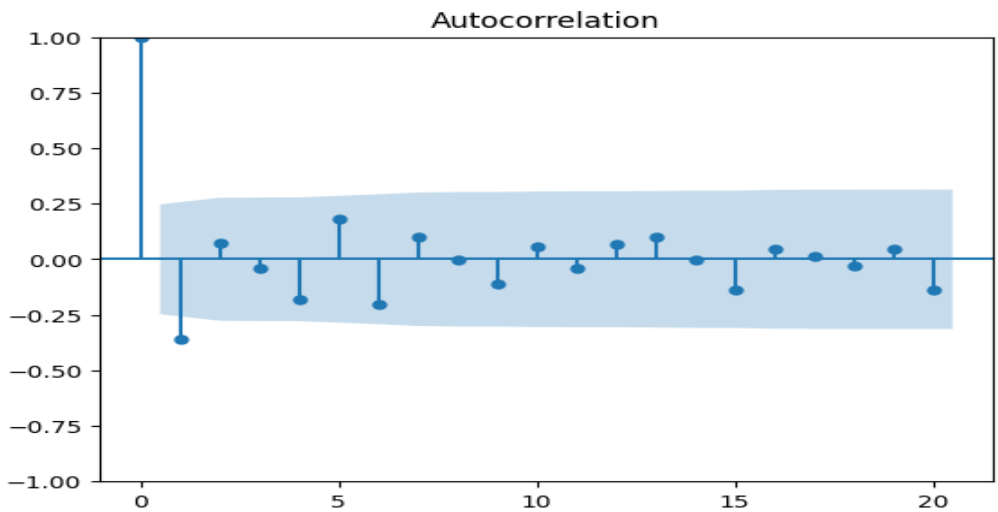

Figure 2: AUTO CORRELATION FUNCTION

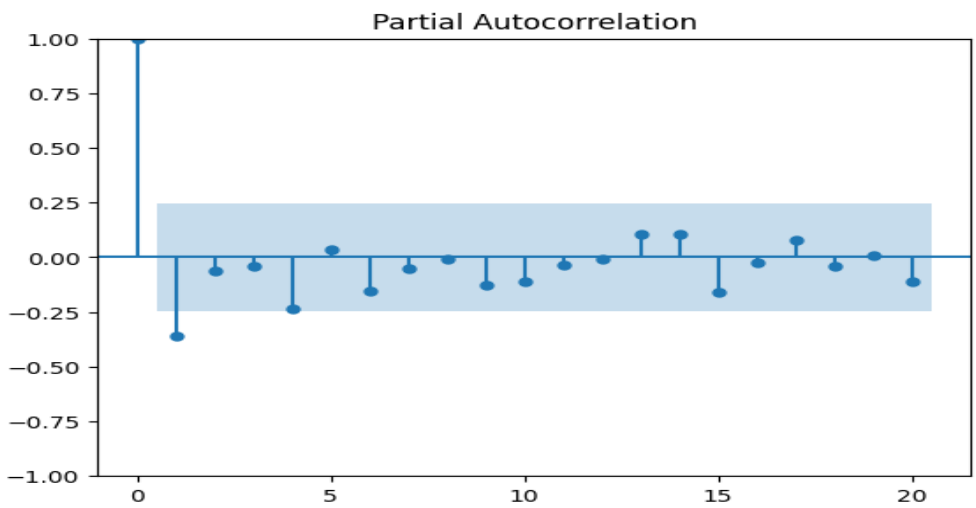

Figure 3: PARTIAL CORRELATION FUNCTION

3.6 Training and Testing:

In this research involved partitioning the dataset into two subsets training set and testing set, to assess the efficacy of the forecasting models. This methodology is crucial in time-series research, since it enables the model to be trained on historical data and subsequently evaluated on novel data to determine its predicted efficiency
ency.

The training dataset consists of the earlier portion of the time-series data and is used to develop the forecasting models. During this phase, the models learn underlying patterns, trends, and relationships present in the data. For instance, models such as ARIMA identify linear dependencies, while LSTM captures more complex and non-linear patterns through its learning process.

The testing dataset contains the latest observations not used in training. This dataset is used to evaluate how well the trained models generalize to new, unseen data. Comparing the predicted values with the actual values in the testing set allows for the evaluation of each model's accuracy and reliability.

In this research, a chronological split was applied, where earlier years were used for training and later years were reserved for testing. This method preserves the temporal order of the data, which is crucial in time-series forecasting, as future values should not influence past observations.
The best approach for training and testing the data is between 80% and 20%.

3.7 Models

3.7.1 Naïve Model

The Naïve Model represents one of the most elementary forecasting methodologies employed in time-series research. It operates under the premise that the future value of a variable corresponds to its latest observed value. The model employs the most recent data point as the forecast for the subsequent time step.
Mathematically, the baseline forecast can be expressed as:

$$y^*(t+1)=y^*t$$

where:

- y^*t is the actual value at time t,
- $y^*(t+1)$ is forecasted value for the next period.

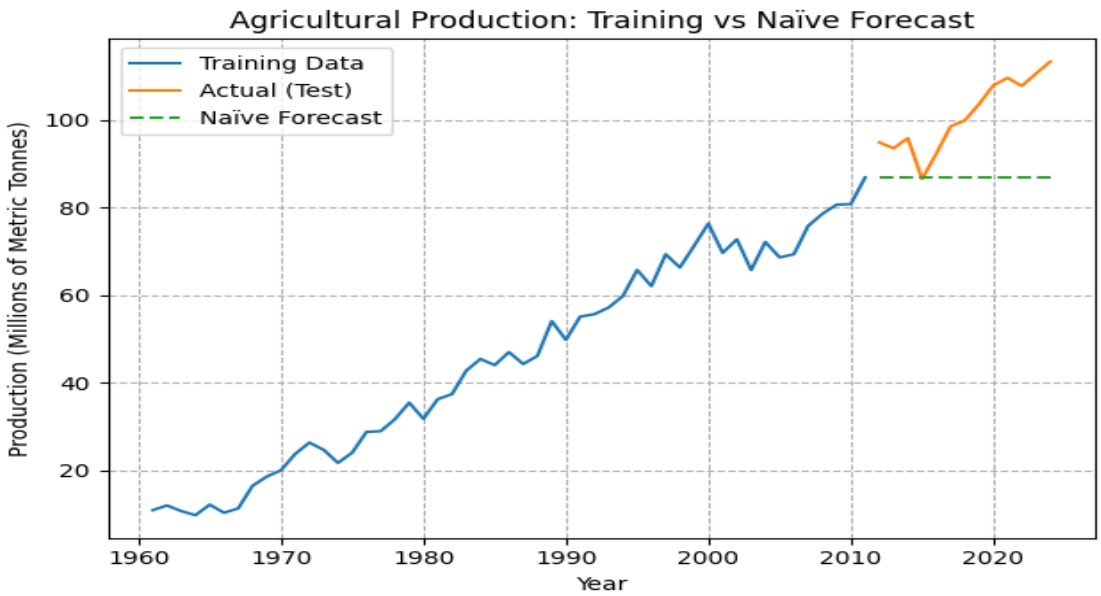

Figure 4: Naïve MODEL

This chart illustrates agricultural production in millions of metric tons from 1960 to approximately 2025, comparing a long-term training dataset against actual test data and Forecasting of baseline.

Observations:

- 1960–2010 (Training Period): Actual production shows a strong long-term upward trend, growing from ~12M tonnes in 1960 to ~85M tonnes by 2010. The growth is consistent but not smooth—year-on-year fluctuations are visible throughout, reflecting real-world agricultural variability, such as weather, policy changes, and seasonal factors.
- 2010 onwards (Test Period): The orange line represents actual production during the test phase. It continues rising but with notable volatility—dipping briefly around 2015 before recovering sharply to ~113M tonnes by 2025.
- Baseline Forecast (green dashed line): This Naïve forecast simply holds the last known training value (~87M tonnes) constant across the entire test period. It does not attempt to model growth or variation, serving purely as a performance benchmark.

The naive prediction does not do a good job of predicting actual production. It greatly underestimates the continuous growth seen in the test period by remaining steady at about 87 million tonnes. By 2025, it will be about 25 million tonnes short. This shows that simple approaches do not work well for datasets with clear rising trends, and it indicates that I need more advanced models (such as ARIMA or LSTM) to keep track of production growth.

The naive model is a point of reference for comparison. I figured out the baseline predictions' performance measures, which include the root mean squared error (RMSE), the mean absolute error (MAE), and the mean absolute percentage error (MAPE). These metrics enable us to ascertain the model's accuracy and establish a minimal performance threshold.

The naïve model is easy to use and doesn't require much processing power, but it has issues. It does not show long-term trends, seasonal fluctuations, or unexpected changes in the data.

Because of this, its ability to anticipate is usually lower than that of more advanced models. But its simplicity makes it a good starting point for any forecasting study.

3.7.2 ARIMA Model

The AutoRegressive Integrated Moving Average (ARIMA) model is a widely used method for predicting the outcome of a time series using statistics. The most effective method of utilising it is to examine and speculate on data that exhibits behaviours and connections that fluctuate over time.
Autoregression (AR), Integration (I), and Moving Average (MA) are the three main parts of ARIMA. Autoregressive shows how an observation is connected to some of its earlier values (lags). In the integration component, the data are differentiated so that they stay in one place, which is needed for time-series modeling to work. Moving average shows how a fact is connected to bad predictions made in the past.

A common way to show the model is as ARIMA (p, d, q), where:
It tells you how many autoregressive terms there are, or how many lags there are in the dependent variable.
d: The amount of differencing that is needed to keep the data from moving
q: how many moving average terms are there (2 times the error term)

Following ADF testing and ACF/PACF analysis, the model was initialized at the order of ARIMA (p, d, q) = (1, 1, 0).

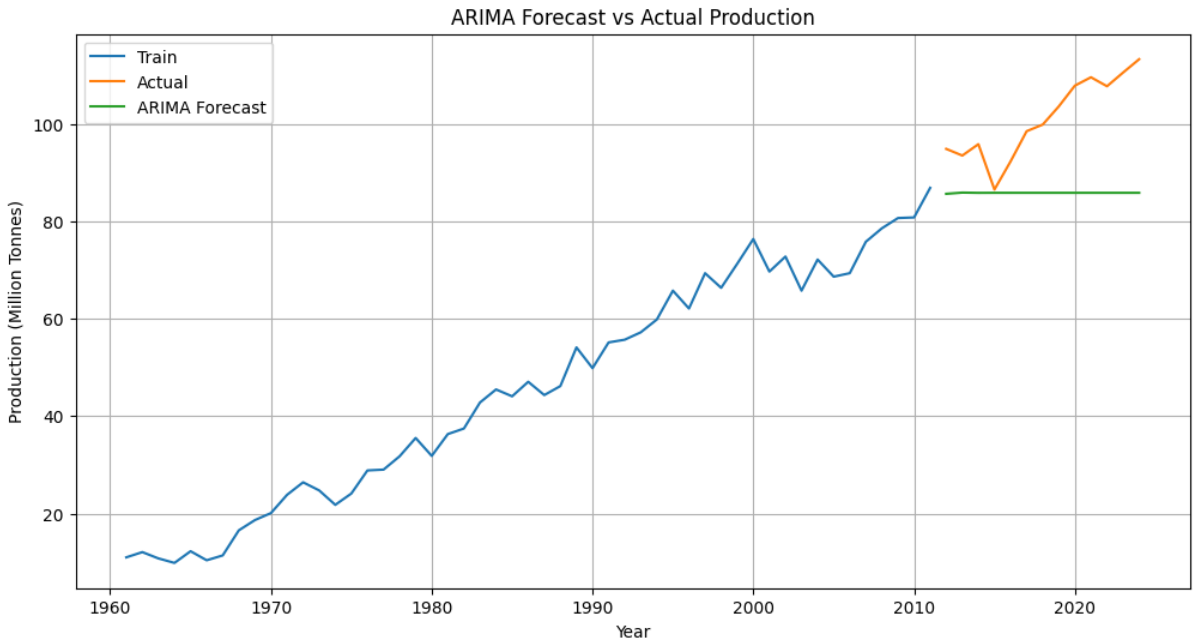

Figure 5: ARIMA Model

The chart shows wheat India produced (in million metric tons) from 1960 to about 2025. Blue line shows training data, orange line shows the actual output during the test period, and the green line shows the ARIMA model's prediction.

Blue line shows training period from 1961 to 2010. The training data reveals a significant and steady rising trend. By 2010, production had grown from about 12 million metric tonnes in 1961 to about 85 million metric tonnes. Growth is steady, but not smooth. Year-on-year changes show how agriculture can alter in the actual world due to things like weather, policy changes, and seasonal influences. The orange line shows the actual production from 2011 to 2025. The real production series keeps going up over the test phase, going from around 95 million metric tonnes in 2012 to about 113 million metric tonnes by 2025. The series has many ups and downs, with a big drop in 2015 followed by a strong recovery.

The ARIMA forecast followed a smooth linear upward trajectory across the test period. While it captured the general direction of growth, it failed to represent the accelerating, nonlinear pace of actual wheat production from 2016 onward, producing forecasts that fell approximately 7–8 million metric tonnes below actual values by 2022. The model achieved an RMSE of 1.72×10^7 and a MAPE of 14.54%, performing marginally worse than even Naïve model, confirming that ARIMA's linear structure is insufficient for this nonlinear dataset.

3.7.2 Hyperparameter Tuning of ARIMA:

This ARIMA model was implemented subsequent to the conversion of the dataset to a stationary format through differencing. Utilising tools such as grid search and performance evaluation based on error metrics, I determined that the optimal values for (p, d, q) were (2, 1, 2).

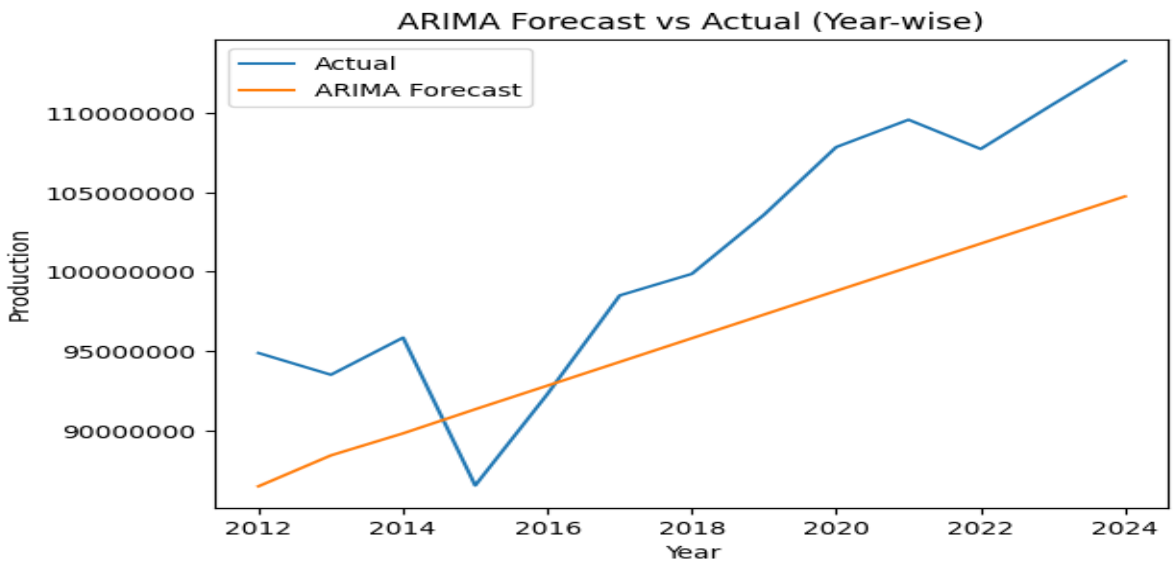

Figure 6: Hyperparameter tuning of ARIMA MODEL

Actual production output against an ARIMA forecasting between 2012 to 2024.
Observations:

- 2012–2015: Actual production starts at ~95M, dips with some volatility, and drops sharply to ~87M in 2015. The ARIMA forecast, by contrast, follows a smooth, steady upward linear trend — it does not capture this real-world volatility at all.
- 2015–2016: Actual production recovers sharply and crosses above the ARIMA forecast line around 2016, where the two briefly converge (~93M).

- 2016–2021: Actual production grows strongly, pulling well ahead of the forecast — reaching ~109M by 2021 while ARIMA predicts only ~98M. The gap widens consistently.
- 2021–2022: A dip in actual production to ~107M, which ARIMA again fails to anticipate.
- 2022–2024: Actual production surges to ~113M, while the ARIMA forecast reaches only ~105M by 2024.

The ARIMA model does not show how production works in real life. Its strictly linear forecast does not take into account any short-term changes and always anticipates output that is lower than what really happened starting in 2016. This shows that ARIMA is not the right model for this dataset because it shows non-linear growth and periodic volatility. This supports the choice of a more advanced model like LSTM.

The model learned from past wheat production data to find trends and make predictions about the future. I looked at its performance using metrics like RMSE, MAE, and MAPE and compared it to other models like the Naïve model and Long Short-Term Memory (LSTM).

- RMSE: 6.56*10^6
- Mape: 6.1*10^6

3.7.3 LSTM Model

An advanced deep learning technique that is specifically designed for the analysis of time-series and sequential data is the Long Short-Term Memory (LSTM) model. Is a class of Recurrent Neural Network (RNN) that is capable of learning long-term dependencies. In situations where historical observations have an impact on ensuing values, this makes it an exceptional choice for forecasting.

I used a MinMaxScaler that was exclusively trained on the training set to normalise the training data to [0, 1]. The normalised series were turned into supervised learning samples via a sliding window function called create_dataset. Each input X has three years' worth of production values (timestep = 3), and the target y is the production value for the next year. The model comprises a single LSTM layer of 50 units with ReLU activation, succeeded by a single Dense output layer. The assembly utilised the Adam optimiser and mean squared error loss. The training persisted for 50 epochs.

The timestep parameter, which is set to 3, is the most critical hyperparameter for the LSTM (Long Short-Term Memory) in this study. Hyperparameters are configuration values that affect how the model learns. It sets the look-back window, which tells the model how much past information it can view before making a forecast. A timestep of 3 gave enough time context to capture short-term production momentum without cutting down on the amount of training samples so much that they could not fit. This parameter has a bigger effect on accuracy than neurone count, epochs, or batch size since it impacts the model's access to information instead of just how quickly it learns.

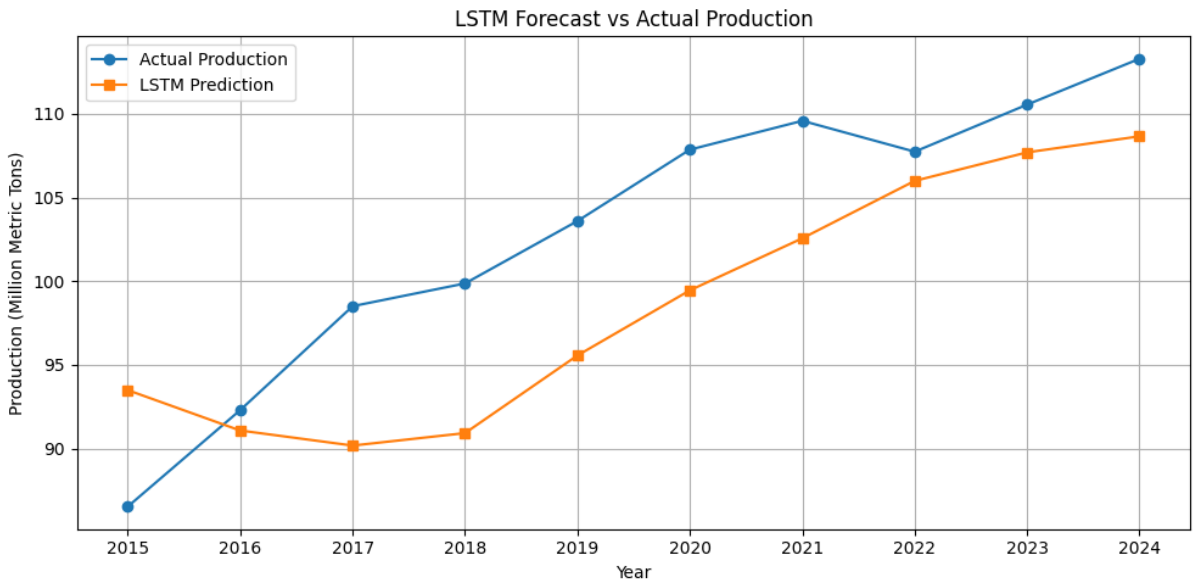

Figure 7: LSTM MODEL

Observations:

- Actual production grew steadily from ~87 (2015) to ~113 (2024), with a minor dip in 2022 (~108) before recovering.
- LSTM predictions started *above* actual in 2015 (~93.5 vs ~87), then consistently underestimated production from 2016 onward.
- The gap between actual and predicted narrowed significantly by 2023–2024, where predictions (~108.5) closely tracked actuals (~110–113).
- Both lines show an overall upward trend, with the model improving in accuracy in later years.

The LSTM results achieved an RMSE of 2.91×10^6 and a MAPE of 8.10%, outperforming both Naïve and ARIMA by a wide margin. The model successfully learned the upward production trend and produced forecasts that converged toward actual values over the test period.

3.7.3. Hyperparameter tuning of LSTM:

I did a manual search over 27 permutations of neurones (20, 50, 100), epochs (20, 50), and batch sizes (1, 4), with timestep = 3 set to stay the same. The conventional LSTM (20 neurones, 50 epochs, batch size 1, RMSE = 4.8×10^6) was used as the base. I tried and tested each configuration on the test set and compared its RMSE to this.

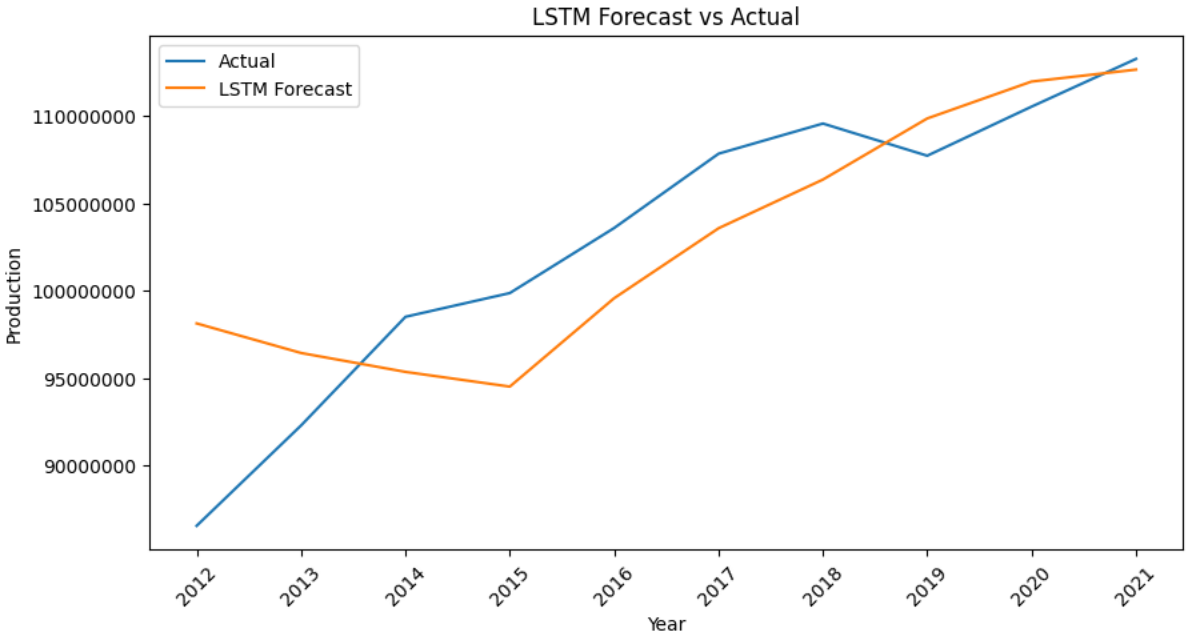

Figure 8: Hyperparameter tuning of LSTM

The best-performing configuration, 20 neurons, 50 epochs, batch size 1 attained an RMSE of 4.80×10^6 and a MAPE of 4.04%, improving on the standard LSTM. This confirms that once the timestep is correctly set, secondary tuning of neurons and epochs provides incremental gains. However, poor choices (too few epochs) can actively degrade accuracy below the standard base, underscoring that systematic hyperparameter evaluation is necessary rather than optional.

3.8 Evaluation Metrics

The effectiveness of the forecasting models was assessed utilising three commonly employed error metrics: Mean Absolute Error (MAE), Mean Absolute Percentage Error (MAPE), and Root Mean Squared Error (RMSE). These metrics provide quantitative evaluations of the divergence between the anticipated and actual observed values.

Root mean squared error (RMSE) is the square root of the mean of the squared differences between predicted and observed values. RMSE is more susceptible to outliers than MAE because the squaring method allocates increased weight to larger mistakes. This attribute renders RMSE especially advantageous in scenarios where substantial errors are unacceptable and demand more stringent punishments. A reduced RMSE (Root Mean Square Error) score signifies enhanced predictive accuracy, similar to MAE (Mean Absolute Error). RMSE is an effective method for identifying models that may exhibit acceptable average performance while occasionally producing significant errors.

The Mean Absolute Error (MAE) tells how big the errors are on average in a group of predictions, no matter which way they go. To find it, you calculate the mean of the absolute differences between the actual and expected values. MAE is easily understood as it presents the error in the same units as the original data. A reduced MAE score signifies superior model performance, as it reflects less discrepancies between anticipated and actual values.

Nevertheless, MAE considers all errors uniformly and does not assign greater significance to larger variances.

The Mean Absolute Percentage Error (MAPE) quantifies the average percentage discrepancy between forecasted and actual production values, providing a comprehensive evaluation of each model's predictive ability. Reduced MAPE (Mean Absolute Percentage Error) ratings, reflecting the precision of the model's forecasts, signify superior model performance.

4: RESULTS

This study's results emphasize the relative efficiency of three forecasting methodologies: the naïve model, the ARIMA model, and the Long Short-Term Memory (LSTM) model. The models were assessed utilizing conventional performance indicators, such as RMSE, MAE, and MAPE.

The naïve model's basic assumption that future values will reflect the most recent observation led to the least accurate results. It did not accurately reflect the fundamental patterns and changes in the dataset, leading to increased error values. This verifies that the dataset has significant temporal patterns necessitating more sophisticated modeling techniques.

The ARIMA model showed a notable enhancement over the naïve model by adeptly capturing linear relationships and temporal dependencies in the data. By employing differencing and optimizing parameters, the model effectively addressed non-stationarity and generated more dependable forecasts. Nonetheless, its efficiency was constrained when addressing intricate nonlinear patterns within the dataset.

Conversely, the LSTM model attained superior overall performance compared to the other two methodologies. Its capacity to understand long-term connections and model nonlinear relationships enabled it to produce more precise and resilient predictions. The incorporation of memory cells and gating methods allowed the model to preserve significant historical data, resulting in reduced error metrics relative to both the naïve and ARIMA models.

The comparison analysis unequivocally illustrates that deep learning methodologies, especially LSTM, are superior for intricate time-series forecasting assignments. Nonetheless, it is crucial to acknowledge that this enhanced performance incurs greater computational complexity and extended training duration.

Moreover, although LSTM offers enhanced accuracy, ARIMA retains significance because of its interpretability and reduced processing demands. Consequently, the selection of a model should be contingent upon the particular application, data attributes, and accessible computational resources. This work shows the importance of choosing suitable models for time-series forecasting and illustrates the benefits of integrating statistical and deep learning methodologies for enhanced predictive efficiency.

Models	**RMSE**	**MAE**	**MAPE**
Naïve	1.629652 * 10^7	1.426131 * 10^7	13.570835
ARIMA	1.7197870 * 10 ^7	1.523864 * 10^7	14.536064
LSTM	2.873898e+06	2.390450e+06	8.13475
Hyper parameter tuning - ARIMA	6.56*10^6	6.1*10^6	5.97400
Hyper parameter tunning - LSTM	4.8*10^6	3.9*10^6	4.03300

5: CONCLUSION AND FUTURE WORK

5.1 Conclusion

The Naïve model, the ARIMA model, and the Long Short-Term Memory (LSTM) model were all used and analyzed in this study as good ways to predict agricultural production. The goal was to see how well they could find trends in time-series data and make accurate predictions.
The outcomes show that naïve model did not perform well because it failed to find trends and deeper patterns in the data. The ARIMA model worked better because it was good at modeling linear interactions and dealing with non-stationarity through differencing. Still, its limitations become clear when dealing with complicated and nonlinear patterns.
The LSTM model attained the highest accuracy among all models. It used to understand long-term dependencies and represent nonlinear interactions, rendering it very adept at time-series forecasting jobs with intricate datasets. This illustrates the capability of deep learning methods to surpass conventional statistical models in predictive analysis.
In this study underscores the significance of model selection in time-series forecasting and validates that machine learning techniques, such as LSTM, can markedly enhance prediction accuracy. These findings can enhance decision-making in agricultural planning and resource management.

5.2 Limitations

The models are unable to understand intricate patterns and relationships in the data due to the limited size of the dataset they employ. The results may be more difficult to apply in larger scenarios as a consequence of the small sample size.
Outer factors like weather, soil types, the economy, and government policies can all have a big effect on agricultural production, but they are not included in this study.
The models only use historical output data because these factors were excluded, which could reduce their accuracy in making predictions. The last thing is that using annual statistics makes it harder to get specific results. It is not possible to account for seasonal changes or short-term changes within a year because the dataset only has records from one year. This would help the accuracy of the predictions if it were possible.

5.3 Future Work

Future studies may concentrate on improving prediction accuracy by integrating supplementary external variables, like meteorological conditions, precipitation, temperature, and economic considerations, which substantially affect agricultural production. Adding these outside factors can help the model better understand the intricacies of the real world.
Additionally, utilising higher-frequency data (e.g., monthly or quarterly instead of annual data) may yield more nuanced insights and enhance predictive accuracy by reflecting short-term fluctuations and seasonal trends.
The creation of hybrid models that combine the best features of statistical methods like ARIMA and deep learning models is another potential area of research. These hybrid models can use both linear and nonlinear patterns, which could make forecasts more accurate and reliable.

REFERENCES

Food and Agriculture Organization, 2024. *FAOSTAT: Statistical Database*. Available at: https://www.fao.org/faostat/

Box, George E. P.. *Time Series Analysis : Forecasting and Control*, John Wiley & Sons, Incorporated,2015. *Pro Quest Ebook Central*, https://ebookcentral.proquest.com/lib/herts/detail.action?docID=7103937

Spyros Makridakis, Steven C. Wheelwright and Rob J Hyndman, 1998. *Forecasting: Methods and Applications*. 3rd ed. New York: John Wiley & Sons. https://go.exlibris.link/BPLMnDbF

Yoo, T.-W.; Oh, I.-S. Time Series Forecasting of Agricultural Products' Sales Volumes Based on Seasonal Long Short-Term Memory. *Appl. Sci.* 2020, *10*, 8169. https://doi.org/10.3390/app10228169

APPENDIX

```
import pandas as pd # this data frame used for reading, filtering, and reshaping tabular data
import numpy as np # numerical operations
import matplotlib.pyplot as pltt # used for plotting the time series
import matplotlib.ticker as tic
from statsmodels.tsa.stattools import adfuller
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf
from statsmodels.tsa.arima.model import ARIMA
from sklearn.metrics import mean_squared_error, mean_absolute_error,r2_score
from sklearn.preprocessing import MinMaxScaler
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense

# storing the location of your CSV file
file_path = "Production_Crops_Livestock_E_All_Data.csv"
df = pd.read_csv(file_path)
print(df.shape)
df.head()

#countries = ["India","United Kindom","United States of America"]
country = "India"
crop = "Wheat"
df_filtered = df[
   (df["Area"] == country) &
   #(df["Area"].isin(countries))&
   (df["Item"] == crop) &
   (df["Element"] == "Production")]
df_filtered.head()

Unit = df_filtered["Unit"].iloc[0]

year_columns = [col for col in df_filtered.columns if col.startswith('Y') and not
(col.endswith('N') or col.endswith('F'))]
ts = df_filtered[year_columns].T
ts.index = [int(''.join(filter(str.isdigit, year))) for year in year_columns]
ts.columns = ["Production"]
ts.head()

# The 'Year' column has already been converted to a datetime index in the previous step.
# Therefore, the following two lines are redundant and cause a KeyError.
ts["Production"] = pd.to_numeric(ts["Production"],errors="coerce")

# Removing missing values
ts = ts.dropna()

# Removing zero or negative values
```

```
ts = ts[ts["Production"] > 0]

# Converting to datetime
ts.index = pd.to_datetime(ts.index, format="%Y")

# Sorting chronologically
ts = ts.sort_index()
print("Cleaned Data Preview:")
print(ts.head())
print("Total Observations:", len(ts))

# plotting the EDA
ts["Production_Million"] = ts["Production"]/1000000
pltt.figure()
pltt.plot(ts.index, ts["Production_Million"])
pltt.title("Wheat Production (Metric Tonnes) of india")
pltt.xlabel("Year")
pltt.ylabel("Production Metric Tonnes")
pltt.grid(True, linestyle='--', alpha=0.6)
pltt.tight_layout()
pltt.show()

#Training-testing 80% train, 20% test
train_size = int(len(ts) * 0.8)
train = ts.iloc[:train_size]
test = ts.iloc[train_size:]
print("Train size:", len(train))
print("Test size:", len(test))

#Naïve (Baseline) forcasting
baseline = np.repeat(train["Production"].iloc[-1],len(test))

#Storing the results in DataFrame
baseline_df = pd.DataFrame({
   "Actual": test["Production"],
   "Baseline Forecast": baseline}, index=test.index)

# plotting the Training, Actual, and Baseline Forecasting
pltt.figure()
pltt.plot(train.index, train["Production"] / 1000000,)
pltt.plot(test.index, test["Production"] / 1000000,)
pltt.plot(test.index, baseline/ 1000000,linestyle="--")
pltt.title("Wheat Production baseline Forecasting")
pltt.xlabel("Years of production")
pltt.ylabel("Production (Metric Tonnes)")
pltt.gca().yaxis.set_major_formatter(tic.StrMethodFormatter('{x:,.0f}'))
pltt.grid(True, linestyle='--')
```

```
pltt.legend()
pltt.tight_layout()
pltt.show()

pltt.figure()
pltt.plot(baseline_df.index, baseline_df["Actual"] / 1000000)
pltt.plot(baseline_df.index, baseline_df["Baseline Forecast"] / 1000000)
pltt.title("Baseline Forecasting")
pltt.xlabel("Year of production")
pltt.ylabel("Production (Millions of Metric Tonnes)")
pltt.gca().yaxis.set_major_formatter(tic.StrMethodFormatter('{x:,.0f}'))
pltt.grid(True, linestyle='--', alpha=0.6)
pltt.legend()
pltt.tight_layout()
pltt.show()


#ARIMA require the time series to have a constant mean and variance over time.
result = adfuller(ts["Production"])
print("ADF Statistic:", result[0])
print("p-value:", result[1])

ts_differ = ts["Production"].diff().drop()
result_differ = adfuller(ts_differ)
print("ADF Statistics after differencing the data:", result_differ[0])
print("p-value after differencing the data:", result_differ[1])

pltt.figure(figsize=(10,4))
plot_acf(ts_differ, lags=20)
pltt.show()

pltt.figure(figsize=(10,4))
plot_pacf(ts_differ, lags=20)
pltt.show()

model = ARIMA(train["Production"], order=(1,1,0))
model_fit = model.fit()
forecast = model_fit.forecast(steps=len(test))
forecast_million = forecast / 1_000_000


pltt.figure(figsize=(12,6))

pltt.plot(train.index, train["Production_Million"])
pltt.plot(test.index, test["Production_Million"])
pltt.plot(test.index, forecast_million)
```

```
pltt.title("Wheat production of ARIMA Forecast")
pltt.xlabel("Year")
pltt.ylabel("Production (Million Tonnes)")

pltt.gca().yaxis.set_major_formatter(tic.StrMethodFormatter('{x:,.0f}'))

pltt.legend()
pltt.grid(True)

pltt.show()

scaler = MinMaxScaler()
trained_scale = scaler.fit_transform(train[["Production"]])
tested_scale = scaler.transform(test[["Production"]])

def create_dataset(data, time_step=3):
    X = []
    y = []
    for i in range(len(data)-time_step):
        X.append(data[i:(i+time_step),0])
        y.append(data[i+time_step,0])
    return np.array(X), np.array(y)

# comparing the previous years
time_steps = 3
X_train, y_train = create_dataset(trained_scale, time_steps)
X_test, y_test = create_dataset(tested_scale, time_steps)

X_train = X_train.reshape(X_train.shape[0], X_train.shape[1], 1)
X_test = X_test.reshape(X_test.shape[0], X_test.shape[1], 1)

lstm = Sequential()
lstm.add(LSTM(50, activation='relu', input_shape=(0,1)))
lstm.add(Dense(1))
lstm.compile(optimizer='adam', loss='mse')
lstm.fit(X_train, y_train, epochs=50, verbose=1)
predictions = lstm.predict(X_test)
predictions = scaler.inverse_transform(predictions)
pred_million = predictions.flatten() / 1_000_000

# Converting actual test values
y_test_val = scaler.inverse_transform(y_test.reshape(-1,1))
actual_million = y_test_val.flatten() / 1_000_000

pltt.figure(figsize=(10,5))

pltt.plot(actual_million)
```

```
pltt.plot(pred_million)

pltt.title("LSTM Forecasting")
pltt.xlabel("Test Years(index values)")
pltt.ylabel("Production (Million Metric Tons)")

pltt.ticklabel_format(style='plain', axis='y')
pltt.gca().yaxis.set_major_formatter(tic.StrMethodFormatter('{x:,.0f}'))

pltt.legend()
pltt.grid(True)
pltt.tight_layout()

pltt.show()

# Getting years matching prediction length
test_years = test.index[-len(y_test):]

# Converting scaled values back to original values
y_test_val = scaler.inverse_transform(y_test.reshape(-1,1))
actual_million = y_test_val.flatten() / 1_000_000
pltt.figure(figsize=(10,5))
pltt.plot(test_years, actual_million)
pltt.plot(test_years, pred_million)
pltt.title("LSTM Forecasting")
pltt.xlabel("Year")
pltt.ylabel("Production (Million Metric Tons)")
pltt.ticklabel_format(style='plain', axis='y')
pltt.gca().yaxis.set_major_formatter(tic.StrMethodFormatter('{x:,.0f}'))

pltt.legend()
pltt.grid(True)
pltt.tight_layout()

pltt.show()

# Baseline
baseline_rmse = np.sqrt(mean_squared_error(test["Production"], baseline))
baseline_mape = np.mean(np.abs((test["Production"] - baseline) / test["Production"])) * 100
baseline_mae = mean_absolute_error(test["Production"], baseline)

# ARIMA
arima_rmse = np.sqrt(mean_squared_error(test["Production"], forecast))
arima_mape = np.mean(np.abs((test["Production"] - forecast) / test["Production"])) * 100
arima_mae = mean_absolute_error(test["Production"], forecast)

# LSTM
```

```
actual_lstm = test["Production"].values[:len(predictions)]
lstm_rmse = np.sqrt(mean_squared_error(actual_lstm, predictions))
actual_lstm = test["Production"].values[:len(predictions)]

lstm_mape = np.mean(np.abs((actual_lstm - predictions) / actual_lstm)) * 100
lstm_mae = mean_absolute_error(actual_lstm, predictions)

# Printing Results
print("Baseline RMSE:", baseline_rmse)
print("Baseline MAPE:", baseline_mape)
print("Baseline MAE:", baseline_mae)
print()
print("ARIMA RMSE:", arima_rmse)
print("ARIMA MAPE:", arima_mape)
print("ARIMA MAE:", arima_mae)
print()
print("LSTM RMSE:", lstm_rmse)
print("LSTM MAPE:", lstm_mape)
print("LSTM MAE:", lstm_mae)

results = pd.DataFrame({

"Model":["Baseline","ARIMA","LSTM"],

"RMSE":[baseline_rmse, arima_rmse, lstm_rmse],
"MAPE":[baseline_mape, arima_mape, lstm_mape],
"MAE":[baseline_mae, arima_mae, lstm_mae]

})

print(results)

# Creating results dataframe of Root Mean Square Error
results = pd.DataFrame({
   "Model": ["Baseline", "ARIMA", "LSTM"],
   "RMSE": [baseline_rmse/1000000, arima_rmse/1000000, lstm_rmse/1000000]
})

print(results)

# Plotting
pltt.figure(figsize=(8,5))

pltt.bar(results["Model"], results["RMSE"])

pltt.title("Models Comparison using RMSE")
pltt.xlabel("Models")
```

```
pltt.ylabel("RMSE in Metric Tonnes")

pltt.grid()
pltt.tight_layout()

pltt.show()

# Creating results dataframe of Mean Absolute Percentage Error
results = pd.DataFrame({
   "Model": ["Baseline", "ARIMA", "LSTM"],
   "MAPE": [baseline_mape, arima_mape, lstm_mape]
})

print(results)

pltt.figure(figsize=(8,5))
pltt.bar(results["Model"], results["MAPE"])

pltt.title("Models Comparison using MAPE")
pltt.xlabel("Models")
pltt.ylabel("RMSE in Metric Tonnes")

pltt.grid(axis='y')
pltt.tight_layout()

pltt.show()

# Creating results dataframe Mean absolute Error
results = pd.DataFrame({
   "Model": ["Baseline", "ARIMA", "LSTM"],
   "MAE": [baseline_mae/1000000, arima_mae/1000000, lstm_mae/1000000]
})

print(results)

# Plotting
pltt.figure(figsize=(8,5))
pltt.bar(results["Model"], results["MAE"])

pltt.title("Models Comparison using MAE")
pltt.xlabel("Models")
pltt.ylabel("RMSE in Metric Tonnes")

pltt.grid(axis='y')
pltt.tight_layout()

pltt.show()
```

```
# Hyperparameter tuning for ARIMA
p_values = [0, 2]
d_values = [0, 1]
q_values = [0, 2]

best_score = float("inf")
best_order = None

for p in p_values:
   for d in d_values:
      for q in q_values:
         try:
            model = ARIMA(train["Production"], order=(p, d, q))
            model_fit = model.fit()

            forecast = model_fit.forecast(steps=len(test))

            # RMSE
            rmse = np.sqrt(mean_squared_error(test["Production"], forecast))

            # MAE
            mae = mean_absolute_error(test["Production"], forecast)

            # MAPE
            mape = np.mean(
               np.abs((test["Production"] - forecast) / test["Production"])
            ) * 100

            print(f"ARIMA({p},{d},{q}) -> RMSE: {rmse:.3f}, MAE: {mae:.3f}, MAPE:
{mape:.3f}%")

            # Using RMSE to selecting best model
            if rmse < best_score:
               best_score = rmse
               best_order = (p, d, q)

         except:
            continue

print("\nBest ARIMA Order:", best_order)
print("Best RMSE:", best_score)

pltt.figure()

pltt.plot(test.index, test["Production"], label="Actual")
pltt.plot(forecast.index, forecast, label="ARIMA Forecast")
```

```
pltt.xlabel("Year")
pltt.ylabel("Production")
pltt.title("Hyperparameter tunning of ARIMA Forecast")

pltt.ticklabel_format(style='plain', axis='y')

pltt.legend()
pltt.show()

# Hyperparameter tuning for LSTM
neurons_lt = [20, 50, 100]
epochs_lt = [20, 50]
batch_siz = [1, 4]

best_rmse = float("inf")
best_params = None

for neurons in neurons_lt:
   for epochs in epochs_lt:
      for batch in batch_siz:

         model = Sequential()
         model.add(LSTM(neurons, input_shape=(time_steps,1)))
         model.add(Dense(1))

         model.compile(optimizer='adam', loss='mse')

         model.fit(X_train, y_train, epochs=epochs, batch_size=batch, verbose=0)

         pred = model.predict(X_test)
         pred = scaler.inverse_transform(pred)

         # RMSE
         rmse = np.sqrt(mean_squared_error(y_test_val, pred))

         # MAE
         mae = mean_absolute_error(y_test_val, pred)

         # MAPE
         mape = np.mean(
            np.abs((y_test_val - pred) / y_test_val)
         ) * 100

         print(f"Neurons:{neurons}, Epochs:{epochs}, Batch:{batch} → RMSE:{rmse:.3f},
MAE:{mae:.3f}, MAPE:{mape:.3f}%")
```

```
        if rmse < best_rmse:
            best_rmse = rmse
            best_params = (neurons, epochs, batch)

print("\nBest LSTM Parameters:", best_params)
print("Best RMSE:", best_rmse)

y_actual = y_test_val.flatten()
y_pred = pred.flatten()

min_val = min(len(y_actual), len(y_pred))

y_actual = y_actual[:min_val]
y_pred = y_pred[:min_val]
x_axis = test.index[:min_val]

pltt.figure(figsize=(10,5))

pltt.plot(x_axis, y_actual)
pltt.plot(x_axis, y_pred)

pltt.xlabel("Year of production")
pltt.ylabel("Production of wheat in india")
pltt.title("Hyperparameter tuning of LSTM Forecast")

pltt.gca().yaxis.set_major_locator(tic.MaxNLocator(nbins=6))
pltt.gca().yaxis.set_major_formatter(tic.StrMethodFormatter('{x:.0f}'))

pltt.legend()
pltt.show()
```